

Problem Statement

A patient walks into a clinic's virtual assistant and starts chatting. They might say "I need to see Dr. Sharma next Tuesday for a headache" or "cancel my appointment" or just "hi, how does this work?"

Your job is to build the brain behind this assistant — a LangGraph application that:

- Understands what the patient wants (book, cancel, reschedule, check status, or just chat)
- Extracts their details from free-text messages (name, doctor, date, reason)
- Checks doctor availability — across multiple doctors at once
- Picks the best slot and presents it for the patient to confirm, edit, or reject
- Detects urgent cases (chest pain, surgery) and pauses for staff approval before booking
- Handles slot conflicts gracefully — if a slot is taken, tries the next one
- Remembers everything across turns — the patient shouldn't have to repeat themselves

The interface is a **CLI chatbot**. The patient types messages, the system responds. That's it. The CLI is a while-loop — all the intelligence lives in the LangGraph workflow.

Here's what a conversation looks like:

```
$ python main.py

=== AI Clinic Booking System ===

[Patient] > Hi, I'm Priya. I need to see Dr. Sharma next Tuesday for a headache.

[System]  Name: Priya | Doctor: Dr. Sharma | Date: May 12 | Reason: Headache
          Checking availability...
          Found 3 slots. Recommended: 10:00 AM

          --- CONFIRM / EDIT / CANCEL ---

[Patient] > confirm

[System]  Booked! ID: BK-20260512-001

[Patient] > I need to cancel that appointment

[System]  Found BK-20260512-001 from your session. Cancelled.

# Urgent case
[Patient] > I have severe chest pain, need a doctor immediately

[System]  URGENT CASE DETECTED. Awaiting staff approval.
```

	Proposed: Dr. Patel, Today 2:00 PM
	(Type 'staff' to switch to staff mode)
[Staff]	> override Today 11:00 AM
[System]	Slot changed to 11:00 AM. Booked! ID: BK-20260508-003

*All doctor schedules and availability data are **mocked** — no real calendar or email integrations. Use **SQLite** for persisting appointments and audit data. The focus is entirely on the LangGraph workflow, not external systems.*

Submit modular code with tests (LLM calls mocked), demo scripts for each milestone, graph visualizations, and a README with setup instructions.

Milestones

Build these in order. Each one layers on top of the previous.

Milestone 1 — State Design & Persistence

Design the complete **BookingState** for the system. It must carry: the full chat history, patient details (name, doctor, date, reason), a list of available slots collected from multiple sources, the confirmed slot, a booking ID, a retry counter, an urgency flag, and an approval flag.

Every field must have a deliberate choice about whether it uses a **reducer** or not. The `available_slots` field is the critical one — demonstrate that it breaks without a reducer and works correctly with one.

Wire up a **checkpoint** and compile a minimal graph (doesn't need to do anything real yet). Prove that state accumulates across multiple invocations on the same thread, and that two different threads never share data. Print the checkpoint history.

Milestone 2 — Tools & Intent Routing

Build **four tools**: check availability for a doctor on a date, book a specific slot (must raise an error if the slot is already taken), cancel a booking by ID, and send a confirmation. All use mock data. The mock must have at least one pre-occupied slot that triggers the booking error.

Build a **supervisor** that classifies the patient's message into one of five intents: **book**, **cancel**, **reschedule**, **status check**, **small talk**. Route each intent to a different branch using conditional edges. Non-smalltalk branches can be stubs for now. Small talk gets a warm reply.

The booking tool's error must be caught and returned as a readable message, not a crash.

Milestone 3 — Agent Integration

Replace the stubs with two real LLM-powered agents, each with its own system prompt and tool set.

The **intake agent** extracts patient details (name, doctor, date, reason) from free text. It has no tools — it only reasons over the conversation and returns structured output.

The **booking agent** receives available slots, picks the best one, books it, and sends a confirmation. It has access only to the booking and confirmation tools.

The availability tool must contain a **nested LLM call**: when a patient says “next Tuesday afternoon,” the tool internally converts it to a specific date before looking up the mock calendar. The outer agent never sees this conversion.

Both agents must be connected to the parent graph through **wrapper functions** that translate between BookingState and each agent’s internal message format.

Milestone 4 — Human-in-the-Loop Gates

Add **two interrupt points** that pause the graph for human input before any irreversible action.

Patient confirmation: After the booking agent selects a slot, the graph pauses and presents the proposed appointment. The patient can confirm (proceed to booking), request a change (graph loops back to the booking agent with updated info), or cancel (graph exits cleanly with no booking).

Staff approval: When the visit reason suggests urgency (chest pain, surgery, emergency), the graph pauses for staff input. Staff can approve the slot as-is, or override it with a different one. The override must use **update_state** to inject the corrected slot from outside the graph, followed by **invoke(None)** to resume.

Demonstrate all four paths: happy (confirm), edit (change then confirm), cancel, and urgent (staff override). Your README must explain why `invoke(None)` is correct after `update_state` but `invoke({})` is correct for a normal multi-turn conversation.

Milestone 5 — Middleware & Observability

Add observability to the existing graph **without changing its topology**.

A **logging/timing decorator** must wrap every node. It records when each node starts, finishes, and how long it took in milliseconds. It must use `functools.wraps` so decorated functions retain their original names and docstrings.

A **retry decorator** must wrap the booking tool specifically. It retries up to three times with exponential backoff when the tool raises its slot-conflict error, then re-raises. Retry attempts must be visible in the log output.

Demonstrate both **stream_mode="updates"** and **stream_mode="values"** on a booking flow. Your README must explain the practical difference between the two and when you'd choose each.

Milestone 6 — Parallelism, Cycles & Subgraphs

Make the availability check run **in parallel across three doctors simultaneously** using `Send`. Each branch contributes to the shared `available_slots` field. A timing comparison must prove that three parallel checks take approximately the same wall-clock time as one.

If the booking agent selects a slot that is already taken, the graph must **loop back** and try again. The retry counter in state increments on each attempt and the loop stops after three retries. Your README must explain what happens if the cycle has no upper bound and how `recursion_limit` provides a safety net. Demonstrate this by intentionally tripping the limit.

Extract the availability logic (date parsing + slot lookup) into a **self-contained subgraph** with its own state schema. It must compile and run independently in tests, then be used as a node inside the parent graph.

Milestone 7 — Full System & Reflection

Connect all milestones into one complete system with the CLI as the single entry point. The supervisor must route using **Python logic only** — no LLM in the supervisor.

Provide four runnable demo scripts:

- **Happy path** — routine booking through the full flow
- **Urgent path** — urgent symptom → staff approval interrupt → slot override → booked
- **Retry path** — first two slots occupied, third succeeds, retry counter visible
- **Multi-turn memory** — same thread: book in turn 1, cancel in turn 2 using the booking ID from the checkpoint

Render the full graph with `xray=True` so subgraph internals are visible.

Answer these four questions in your README:

- If `check_availability` called a real calendar API that occasionally times out, which milestone's work would handle it and what would you change?
- Why is `invoke({})` correct for a returning patient but `invoke(None)` correct after `update_state`?
- What concretely breaks in the happy path if you remove the reducer from `available_slots`?
- Which single change from any milestone had the biggest impact on your ability to debug the system, and why?

Answers must reference your specific code. Generic answers receive no credit.

Brownie Points

- **WebSocket interface** — a FastAPI WebSocket endpoint as an alternative to the CLI
- **Streamlit or Chainlit UI** — a minimal chat UI wrapping your graph
- **LangSmith tracing** — full tracing for all agent and tool calls
- **Postgres checkpointer** — replace MemorySaver with persistent checkpointing
- **Full rescheduling flow** — cancel old + book new in a single conversation turn
- **Multi-patient concurrency** — two patients with different threads running simultaneously

The CLI is just a while-loop. The graph is the product.